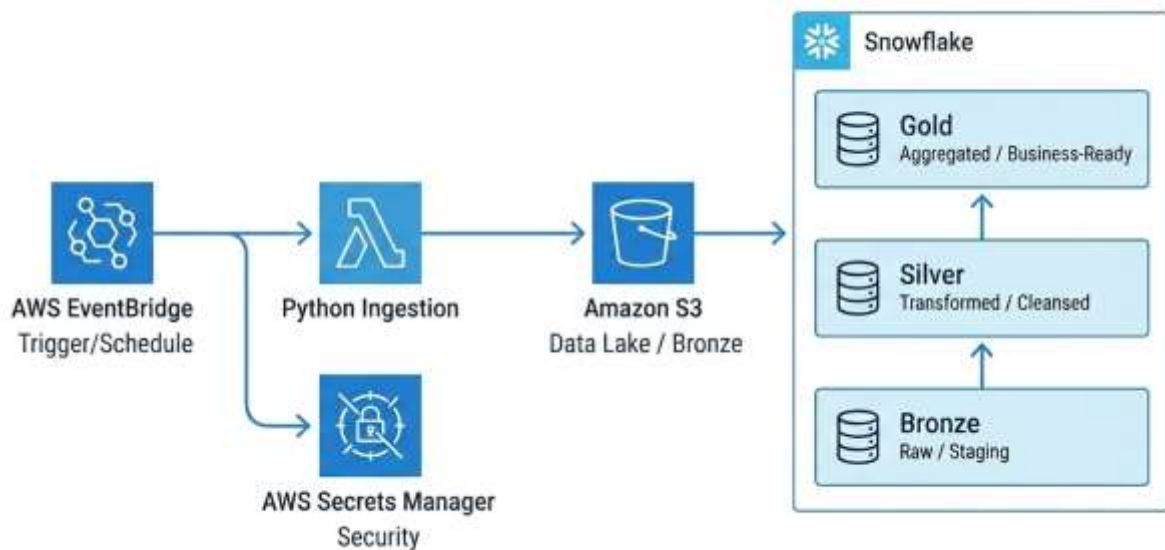


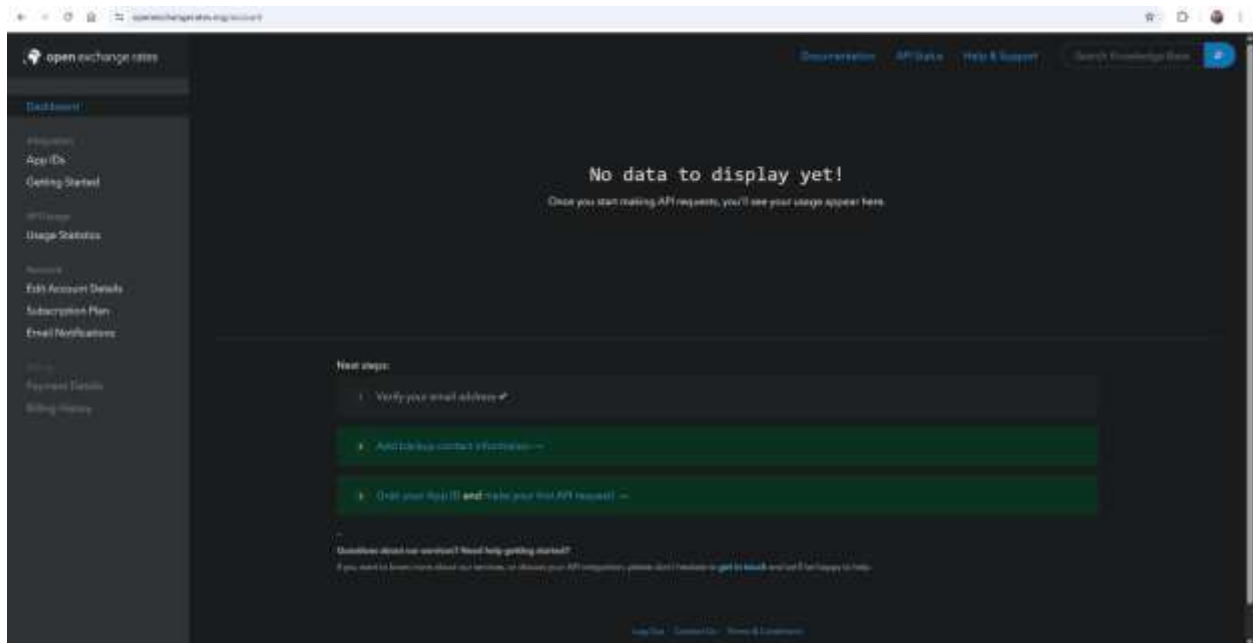
# SNOWFLAKE S3 LAMBDA EVENTBRIDGE RULE PROJECT

## OPEN CURRENCY EXCHANGE RATE

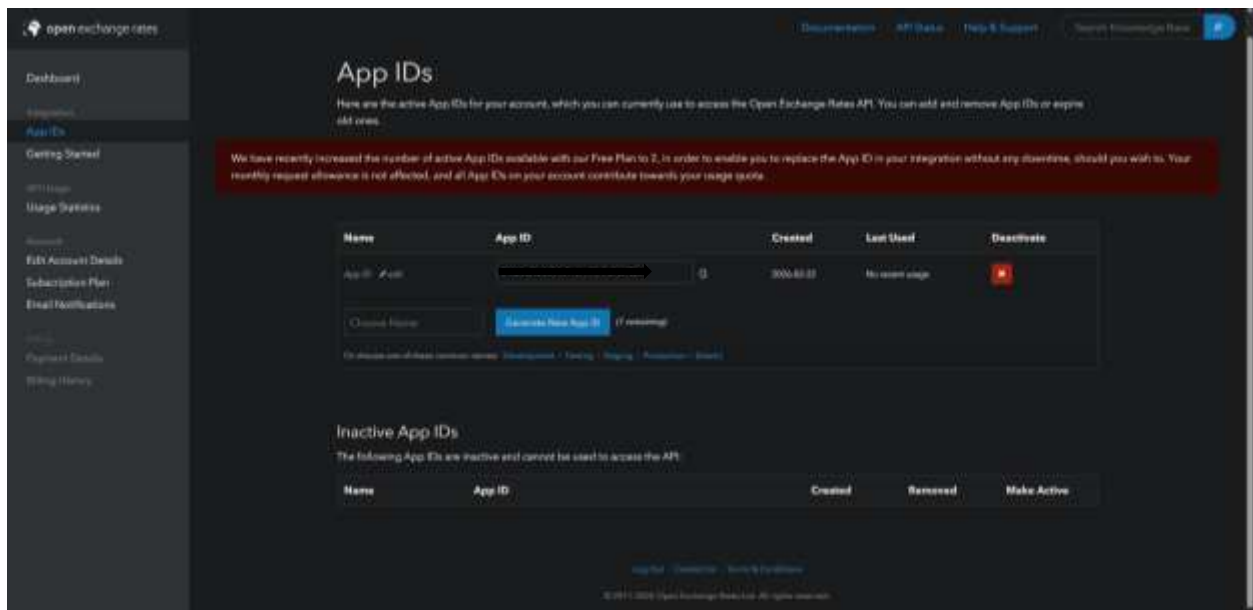


# Create account on Open Currency Exchange Rate

<https://openexchangerates.org/account>



Click on **App IDs**



Copy your **APP ID**

# SNOWFLAKE SETUP

## Step1: (Database & Tables)

Snowflake mein login kren aur ye SQL commands run kren ta k data receive krne k liye jaga bn jae

```
-- 1. Create Database and Schema
CREATE DATABASE IF NOT EXISTS CURRENCY_DB;
CREATE SCHEMA IF NOT EXISTS CURRENCY_DB.CURRENCY;

-- 2. Use Database and Schema
USE DATABASE CURRENCY_DB;
USE SCHEMA CURRENCY;

-- 2. Create Raw table Where JSON data will store
CREATE OR REPLACE TABLE CURRENCY_DB.CURRENCY.EXCHANGE_RATES_RAW (
  RAW_DATA VARIANT,
  LOAD_TIMESTAMP TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  FILE_TIMESTAMP STRING
);
```

## Step 2: Stored Procedure Create Karein

Ek procedure ko Snowflake mein create karna hoga jo JSON data ko table mein insert kare.

```
CREATE OR REPLACE PROCEDURE SP_EXCHANGE_RATE_LOADING(JSON_INPUT STRING, TS_INPUT STRING)
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN

  -- We use direct SELECT, so that PARSE_JSON result will become Variant
  INSERT INTO EXCHANGE_RATES_RAW (RAW_DATA, FILE_TIMESTAMP)
  SELECT PARSE_JSON(:JSON_INPUT), :TS_INPUT;

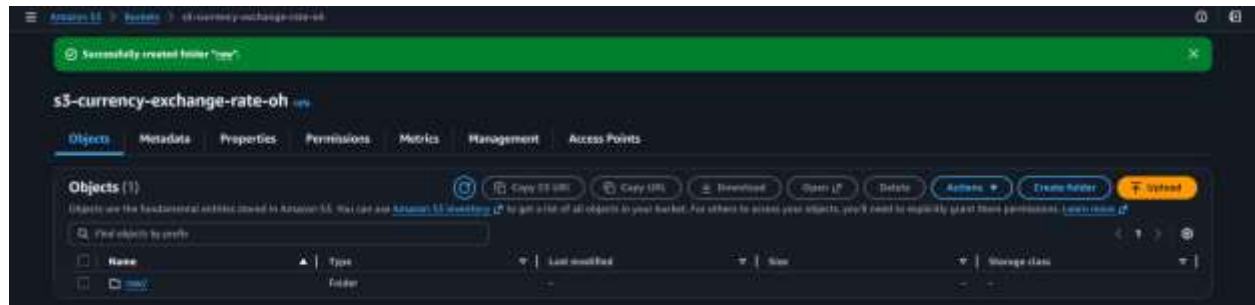
  RETURN 'Success: Data Loaded to Snowflake';

END;
$$;
```

## CREATE S3 BUCKET FOR CURRENCY EXCHANGE RATE:

s3-currency-exchange-rate-oh

Create Folder “raw” in this Bucket



## CREATE LAMBDA FUNCTION

Function name: fn-currency-exchange-rate-oh

Runtime: Python 3.11

Architecture: x86\_64



### Add Trigger:

Click on **Function name** (fn-currency-exchange-rate-usama) -> Add Trigger

Source: [EventBridge \(CloudWatch Events\)](#)

Rule: [Create a new rule](#)

- Rule name: [rule\\_currency-exchange-rate-usama](#)
- Rule Description: [To run the lambda after every one hour](#)
- Rule type: [Schedule Expression](#)
- Schedule Expression: [rate\(1 hour\)](#)

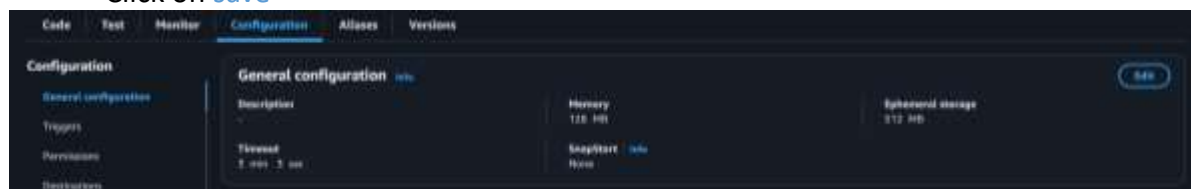
Click on [Add](#)

### Edit Timeout:

Configuration -> General Configuration -> Edit

- Timeout: [3 min 3 sec](#)

Click on [save](#)



## Add Role:

Configuration -> Permissions -> Execution Role → Click on your **Role name**



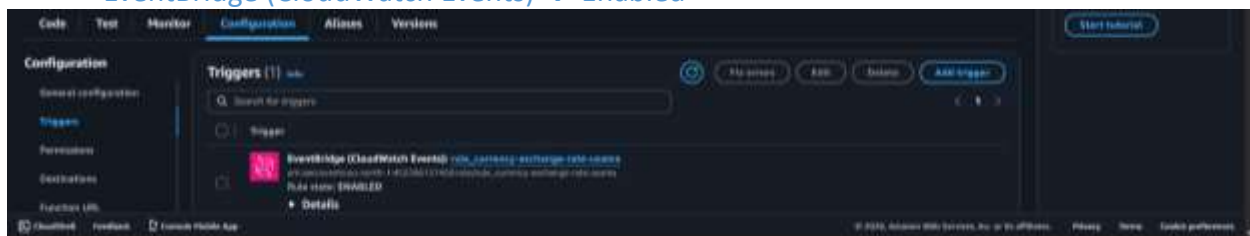
Permissions -> Permission Policies -> Add permissions -> Attach policies

- [AmazonS3FullAccess](#)
- [AwsLambda\\_FullAccess](#)
- [AmazonEventBridgeFullAccess](#)
- [SecretsManagerReadWrite](#)

Click on Add Permissions

## For Testing: Disabled EventBridge (CloudWatch Events)

Lambda -> Functions -> (fn-currency-exchange-rate-oh) -> Configuration -> Triggers  
[EventBridge \(CloudWatch Events\)](#) → Enabled



Click on [rule-currency-exchange-rate-usama](#) → Disable

### 1. Disable Karne Se Kya Hua? (Current State)

Jab aapne EventBridge (CloudWatch Events) ko **Disable** kiya, to aapne Lambda ka "Automatic Switch" band kr diya.

- **Ab kya hoga:** Aapka Lambda function khud se kabhi nahi chalega.
- **Faida:** Aapke AWS aur Snowflake ke fuzool paise (credits) nahi kharch honge.
- **Nuksaan:** Aapko har baar data fetch karne ke liye manual **Test** button dabana padega.

### 2. Enable Karne Se Kya Hoga? (Automation)

Agar aap isay **Enable** kar dete hain, to ye ek "Alarm Clock" ki tarah kaam karega.

- **Ab kya hoga:** Jo bhi time aapne set kiya hoga (maslan har roz raat 12 baje), EventBridge Lambda ko ek signal bhejega aur Lambda **khud-ba-khud** API se data utha kar Snowflake mein daal dega.
- **Faida:** Aapko computer khol kar button dabane ki zaroorat nahi. Aap so rahe honge aur aapka data pipeline kaam kar raha hoga.
- **Nuksaan:** Agar API ya Snowflake mein koi tabdeeli aayi aur function fail hua, to wo background mein fail hota rahega jab tak aap logs check na karein.

## Edit `lambda_function.py` file

Lambda -> Functions -> ([fn-currency-exchange-rate-oh](#)) -> Code → `lambda_function.py`

Delete all and paste this code

```
import os
import json
import boto3
import requests
from snowflake_provider import Provider
from datetime import datetime, timedelta, timezone

def connect_to_snowflake():
    """
    Building connection with snowflake using custom provider class.
    """
    params = {
        'region_name': os.environ['region_name'],
        'aws_db_creds_secret_id': 'db/currency-exchange-rate',
        'aws_db_creds_secret_value': 'fusion_snowflake',
        'snowflake_db': os.environ['snowflake_db'],
        'snowflake_role': os.environ['snowflake_role'],
        'snowflake_wh': os.environ['snowflake_wh'],
        'environment': os.environ['environment']
    }
    provider = Provider(**params)
    return provider

def s3_client(json_data, timestamp):
    """
    Dumping response to S3 as json file.
    """
    dt = datetime.strptime(timestamp, '%Y-%m-%d %H:%M:%S')
    year = dt.strftime('%Y')
    month = dt.strftime('%m')
    day = dt.strftime('%d')
    hour = dt.strftime('%H')

    s3_key = f"exchange_rates/{year}/{month}/{day}/exchange-rates-{hour}.json"
    s3_bucket_name = os.environ.get('s3_bucket_name')

    s3 = boto3.client("s3")
    s3.put_object(Bucket=s3_bucket_name, Key=s3_key, Body=json_data)

def insert_exchange_rates_to_snowflake():
    """
    This function is performing following tasks.
    1. Getting data from API.
    2. Calling s3_client function to dump data into S3
    3. Dumping data to Snowflake
    """
```

```

provider = connect_to_snowflake()
base_url = os.environ.get("oer_base_url")
app_id = os.environ.get("oer_app_id")
base_currency = os.environ.get("oer_base_currency", "USD")
url = f"{base_url}?app_id={app_id}&base={base_currency}"
response = requests.get(url)
if response.status_code == 200:
    data = response.json()
    timestamp = datetime.fromtimestamp(data['timestamp'], tz=timezone.utc).strftime('%Y-%m-%d %H:%M:%S')
    database = "CURRENCY_DB"
    schema = "CURRENCY"
    table_name = "EXCHANGE_RATES_RAW"
    json_data = json.dumps(data)
    s3_client(json_data,timestamp)
    query2 = f"""
    CALL {database}.{schema}.SP_EXCHANGE_RATE_LOADING(%s, %s);
    """
    provider.exe_query(query2, (json_data,timestamp,))
else:
    raise Exception(f"API request failed with status code {response.status_code}")

def lambda_handler(event, context):
    insert_exchange_rates_to_snowflake()
    return {'statusCode': 200}

```

Click on **Deploy** OR Press **Shift+ U + I**

**Create New File: snowflake\_provider.py**

```

import os
import json
import boto3
from snowflake import connector

class Provider:
    def __init__(self, **params):
        """
        Initialize Provider class
        """

        global app_id
        # Initialize variables
        self.region_name = params['region_name']
        self.aws_db_creds_secret_id = params['aws_db_creds_secret_id']
        self.aws_db_creds_secret_value = params['aws_db_creds_secret_value']
        self.snowflake_db = params['snowflake_db']
        self.snowflake_role = params['snowflake_role']
        self.snowflake_wh = params['snowflake_wh']
        # self.schema = schema
        # Get boto3 client, db credentials
        self.client = self.get_client()
        self.db_creds = self.get_db_creds()
        # Set db credentials
        self.username = self.db_creds['username']
        self.password = self.db_creds['password']
        self.account_name = self.db_creds['account_name']
        app_id = self.db_creds.get('app_id')

```

```

def get_client(self):
    """
    Create a boto3 client for secretsmanager
    """
    session = boto3.session.Session()
    client = session.client(service_name="secretsmanager", region_name=self.region_name)
    return client

def get_db_creds(self):
    """
    Get db credentials from AWS secret manager
    """
    get_secret_value_response = self.client.get_secret_value(SecretId=self.aws_db_creds_secret_id)
    db_creds = json.loads(get_secret_value_response['SecretString'])[f'{self.aws_db_creds_secret_value}']
    return db_creds

def get_creds_from_secret(self, secret_id):
    """
    Get credentials from AWS secret manager
    """
    get_secret_value_response = self.client.get_secret_value(SecretId=secret_id)
    creds = json.loads(get_secret_value_response['SecretString'])
    return creds

def get_snowflake_conn(self):
    """
    Get snowflake connection
    """
    params = dict(
        user=self.username,
        password=self.password,
        account=self.account_name,
        role=self.snowflake_role,
        warehouse=self.snowflake_wh,
        database=self.snowflake_db,
    )
    conn = connector.connect(**params)
    return conn

def get_config(self, schema):
    """
    Get config table values from snowflake database
    """
    sql_query = f"""select name, value from {self.snowflake_db}.{schema}.CONFIG where type in ('sql','variable') and
active_flag = TRUE;"""
    self.conn = self.get_snowflake_conn()
    with self.conn as ctx:
        cur = ctx.cursor()
        cur.execute(sql_query)
        df_get_sqls = cur.fetch_pandas_all()
        self.conn.close()
    # Creating environment variables
    for index, row in df_get_sqls.iterrows():
        os.environ[row['NAME']] = row['VALUE']
    return df_get_sqls

```



```

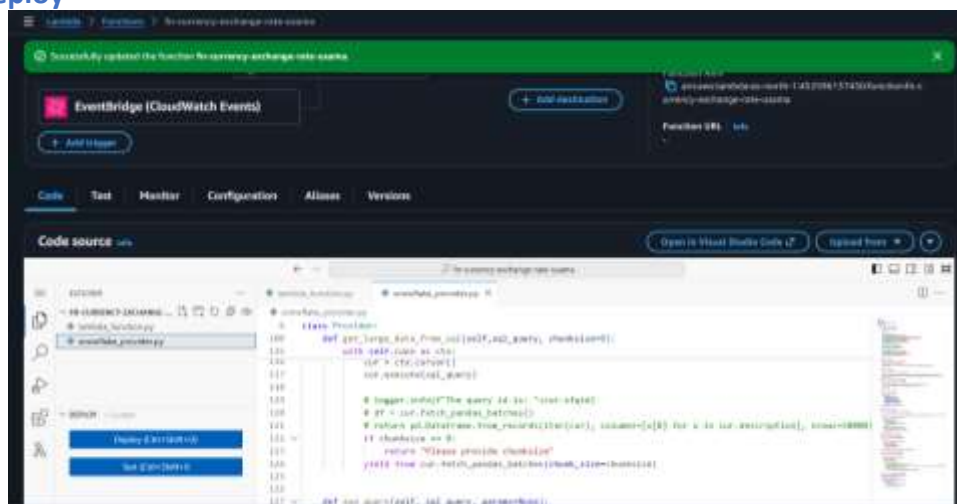
def get_data_from_sql(self,sql_query):
    """
    Get data from snowflake database
    (Execute DQL query using connection)
    """
    self.conn = self.get_snowflake_conn()
    with self.conn as ctx:
        cur = ctx.cursor()
        cur.execute(sql_query)
        df = cur.fetch_pandas_all()
        return df

def get_large_data_from_sql(self,sql_query, chunksize=0):
    """
    Get data from snowflake database
    (Execute DQL query using connection)
    """
    self.conn = self.get_snowflake_conn()
    with self.conn as ctx:
        cur = ctx.cursor()
        cur.execute(sql_query)
        # logger.info(f"The query id is: "+cur.sfqid)
        # df = cur.fetch_pandas_batches()
        # return pd.DataFrame.from_records(iter(cur), columns=[x[0] for x in cur.description], nrows=50000)
        if chunksize == 0:
            return "Please provide chunksize"
        yield from cur.fetch_pandas_batches(chunk_size=chunksize)

def exe_query(self, sql_query, params=None):
    """
    Execute DDL / DML query using connection
    Supports parameter binding
    """
    self.conn = self.get_snowflake_conn()
    with self.conn as ctx:
        cur = ctx.cursor()
        if params:
            cur.execute(sql_query, params)
        else:
            cur.execute(sql_query)

```

Click on **Deploy**



## Create Environment variable

Lambda -> Functions -> (fn-exchange...) -> Configuration -> Environment variables → Edit  
Add all following Key Values

| Key               | Value                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| environment       | DEV                                                                                                       |
| oer_app_id        | YOUR-APP_KEY                                                                                              |
| oer_base_currency | USD                                                                                                       |
| oer_base_url      | <a href="https://openexchangerates.org/api/latest.json">https://openexchangerates.org/api/latest.json</a> |
| region_name       | Us-east-1 (US N. Verginia)-> Region name of Amazon                                                        |
| s3_bucket_name    | s3-currency-exchange-rate-oh                                                                              |
| snowflake_db      | CURRENCY_DB                                                                                               |
| snowflake_role    | ACCOUNTADMIN                                                                                              |
| snowflake_wh      | COMPUTE_WH                                                                                                |

Click on **Save**

## ADD SECRET MANAGER

Aws console -> Search (Secret manager) -> Store a new secret

Secret type: **Other type of secret**

Key/value pairs -> **Plaintext**

Remove rest and Copy/paste this JSON

```
{
  "fusion_snowflake": {
    "host": "https://NFXNRIQ-EDB24896.snowflakecomputing.com/",
    "username": "OSAMAHASHMI28",
    "password": "*****",
    "account_name": "NFXNRIQ-EDB24896",
    "db": "CURRENCY_DB",
    "app_id": "*****"
  }
}
```

Click on **Next**

**Secret name -> ( lambda\_functions.py → params → Copy Value of "aws\_db\_creds\_secret\_id" )**

Click on **Next → Next → Store → View details**

## LAMBDA LAYER

Lambda Layers isliye zaruri hain q k AWS Lambda k standard environment mein **requests** aur **snowflake-connector** pehle se majood nahi hote. Inke bagher apka code ModuleNotFoundError dega.

Custom layer banane ka sahi tareeqa ye hai:

## Step 1: EC2 Instance Launch Karen

1. Ek chota sa [EC2 instance](#) launch karen
2. [t3.micro](#) machine kafi hai.
3. OS: [Amazon Linux 2023](#) select karein.
4. Apni Key bana k attach kren
5. Advanced details mein IAM Role k zrye ye permission den: [AmazonS3FullAccess](#)
6. Instance bn'ne k bad us instance ki details se [Public IP Address](#) copy kren.
7. Us key Folder k andr apne VS Code Terminal open kren
8. SSH k zariye connect ho jayein.

```
ssh -i "my-key.pem" ec2-user@Your Public IP Address
```

## Step 2: Dependencies Install Karen

### 1. System Update aur Pip Installation

Sabse pehle humne Pip install karinge kyunki EC2 par wo mojud nahi hai:

```
sudo dnf install python3-pip -y
```

Pip ko latest version par upgrade karinge (taake new packages mil saken)

```
python3 -m pip install --upgrade pip
```

### 2. Workspace aur Layer ki Taiyari

Purani files delete karke ek saaf new folder structure banaen ta k Lambda ko sahi path mil sake:

```
rm -rf python snowflake_layer_v2.zip  
mkdir -p python
```

### 3. Compatible Libraries ki Installation

Ye sb se important command hai jo Lambda (Python 3.11) k liye sahi binaries download karega:

Snowflake connector aur Requests ko Lambda-compatible format mein install karen

```
pip3 install \  
  --platform manylinux2014_x86_64 \  
  --target=python \  
  --implementation cp \  
  --python-version 3.11 \  
  --only-binary=:all: --upgrade \  
  snowflake-connector-python requests
```

#### 4. Cleanup aur Zipping

Layer ka size kam rakhne ke liye humne fuzool files delete kin aur phir zip banaya:

Pycache files delete karne ke liye

```
find python -name "__pycache__" -type d -exec rm -rf {} +
```

Pure folder ko zip mein convert karne ke liye

```
zip -r snowflake_layer_v2.zip python/
```

#### 5. S3 Bucket mein Upload

Aakhri step mein hum file ko S3 bucket mein bhejinge taake Lambda wahan se utha sake:

Zip file ko apni S3 bucket mein copy karne ke liye

```
aws s3 cp snowflake_layer_v2.zip s3://s3-currency-layer/
```

Ab aapki file [snowflake\\_layer\\_v2.zip](#) S3 bucket ([s3-currency-layer](#)) mein maujood hai, to [Lambda Layer](#) add karne ka kaam bahut aasaan hai. Neeche diye gaye steps follow karein:

---

**Tip:** Jab bhi aap EC2 par ye kaam khatam kar len, to **EC2 instance ko Stop ya Terminate** karna mat bhooliye ga taake fuzool charges na paren.

---

#### Step 3: S3 URL Copy Karein

1. **S3 Console** mein jayein aur apne bucket [s3-currency-layer](#) ko open karein.
2. [snowflake\\_layer\\_v2.zip](#) file k left box par tick karein.
3. Right side pr upar ek button hoga **"Copy S3 URI"**, us par click karein.  
[s3://s3-currency-layer/snowflake\\_layer\\_v2.zip](#)

#### Step 4: Lambda Layer Create Karein

1. **Lambda Console** mein jayein. Left side menu mein **Layers** par click karein.
2. **Create layer** button dabaen.
3. **Name:** [snowflake-connector-layer](#)
4. **Description:** Snowflake and Requests libraries.
5. **Methods:** Yahan **"Upload a file from Amazon S3"** wala option select karein.
6. **Amazon S3 link URL:** [s3://s3-currency-layer/snowflake\\_layer\\_v2.zip](#)
7. **Compatible runtimes:** [Python 3.11](#)
8. **Compatible architectures:** [x86\\_64](#)
9. **Create** par click karein.

## Step 5: Lambda Function mein Layer Add Karen

1. Apne Lambda function [fn-currency-exchange-rate-oh](#) ko open karein.
2. Bilkul niche scroll karein jahan **Layers** ka section hai.
3. Click on **Add a layer**
4. **Layer source:** Select "[Custom layers](#)"
5. **Custom Layers:** Select your custom layer [snowflake-connector-layer](#)
6. **Version:** 2 or 1
7. **Add** par click kar dein.

*Layer attach krne k bad ab Verification (Testing) kren.*

## Step 6: Verification (Testing)

Layer add krne k bad, Lambda k **Code** tab mein jaen aur [lambda\\_function.py](#) file pe **Test** button dabaen.

Agr koi error nahi ae to Snowflake mein [CURRENCY\\_DB.CURRENCY](#) mein ye command run kr k check kren.

```
SELECT * FROM CURRENCY_DB.CURRENCY.EXCHANGE_RATES_RAW;
```



**Snowflake mein data successfully load ho gaya hai.** EXCHANGE\_RATES\_RAW table mein aapki row nazar aa rahi hai jisme base currency "USD" aur timestamp bilkul sahi save hua hai.

Aapne jo data ingest kiya hai, wo **Open Exchange Rates API** se aaya hua real-time currency exchange data hai. Aapke Snowflake mein RAW\_DATA column ke andar jo JSON nazar aa raha hai, usme ye malumaat chhupi hui hain:

### 1. Base Currency

Aapka base rate **USD** (United States Dollar) par set hai. Iska matlab hai ke baqi tamam currencies ke rates 1 USD ke muqable mein dikhae gae hain.

## 2. Exchange Rates (The Core Data)

JSON ke "rates" section mein dunya bhar ki currencies ke live rates maujood hain. Maslan:

- **PKR:** 1 USD ke muqablay mein Pakistani Rupee ki qeemat.
- **INR:** Indian Rupee ka rate.
- **EUR:** Euro ka rate.
- Aapke ek hi "Test" run se saari dunya ki currencies ka data ek saath Snowflake mein chala gaya hai.

## 3. Metadata (Extra Information)

- **Disclaimer:** API ke use ki legal terms.
- **License:** Data provide karne wale ka license information.
- **Timestamp (FILE\_TIMESTAMP):** Wo waqt jab API ne ye rates generate kiye (aapki screenshot mein ye 2026-03-05 14:00:00 hai).

## 4. System Columns (Aapke Table ki Pehchan)

Aapne data ke saath do mazeed cheezein save ki hain:

- **LOAD\_TIMESTAMP:** Ye wo waqt hai jab aapke Lambda function ne data Snowflake ke table mein insert kiya (2026-03-05 06:04:23.491).
- **RAW\_DATA (Variant):** Ye poora ka poora JSON block hai jo Snowflake ne parse karke apne paas mehfooz kar liya hai.